# Lethe: Verifiable Hard Deletion for GDPR-Compliant RAG Vector Databases

Devyani Bishnoi
*Dept. of Computer Science and Engineering (Information Security)*
*Vellore Institute of Technology*
Vellore, India
devyanibishnoi04@gmail.com

Anshika Agarwal
*Dept. of Computer Science and Engineering*
*Vellore Institute of Technology*
Vellore, India
anshikaagarwal740@gmail.com

Hridya Sharma
*Dept. of Computer Science and Engineering (AI and Machine Learning)*
*Vellore Institute of Technology*
Vellore, India
hridyaasharmaaa@gmail.com

***Abstract*—Retrieval-augmented generation (RAG) systems keep private documents as embeddings in vector databases, yet most vector stores answer a deletion request by flagging the record, which leaves a recoverable embedding in storage and gives the data owner no way to verify that erasure happened. We present Lethe, a deletion layer on PostgreSQL and pgvector that (i) partitions the vector table by tenant and, for each deletion, removes the row together with a signed audit entry and then rewrites that partition with `VACUUM FULL`, which also rebuilds its HNSW and IVFFlat indexes, (ii) records every deletion in an ECDSA-signed log whose entries include the previous entry's hash, so that altered entries and entries removed from the middle are detectable, and (iii) erases a data subject across all partitions and issues a verifiable certificate. We test physical erasure by searching the database's own data files for a deleted embedding's exact bytes. Three tested versions of the deletion path gave different results. Rebuilding the indexes inside the deleting transaction left the embedding in the index files in 13 of 20 trials. Rebuilding after the commit with a plain `VACUUM` cleared the indexes but left the heap copy every time. The final path left it in no data file we searched in 20 of 20 trials, although the write-ahead log still holds it. It cost 1 to 21 percent more than the earlier path, 106 ms for a single deletion at 100 documents and 499 ms at 5,000, and both paths stalled concurrent readers for about 0.4 s at 5,000 documents. Rebuilding one 500-document tenant's index took 52 to 71 ms whether 2 or 16 tenants shared the system, against 241 to 825 ms for one combined index. Removing a middle audit entry broke the chain as designed. We report the remaining limits in full.**

***Index Terms*—vector databases, retrieval-augmented generation, right to erasure, GDPR, hard deletion, tamper-evident audit log, pgvector, HNSW**

## I. Introduction

Enterprises that adopt Retrieval-Augmented Generation (RAG) [1] keep their private documents in a vector database and retrieve them at query time, so that a language model can answer from company data without being trained on it. When a person exercises the right to erasure under Article 17 of the GDPR [2], the data must disappear from that vector store, not only from the original source files.

Most vector stores do not fully do this. A delete call usually flags a record as removed while the embedding stays in the index files, and a 2026 analysis of soft-delete behaviour across several HNSW implementations showed that deleted embeddings can be reconstructed directly from storage, bypassing the database's own access controls [3]. A provider that tells a client "your data is deleted" may therefore still hold a recoverable copy of it. For privately hosted, enterprise LLM services, where verifiable deletion is part of the trust promise, this gap is a direct compliance risk.

Two obstacles stand in the way of closing it. First, removing a vector from a graph or cluster index for real usually means rebuilding the whole index, which is too expensive for large knowledge bases that change often. Second, even when a deletion is carried out, the client has no way to check it afterwards: an ordinary log can be edited by anyone with database access, so a missing entry proves nothing.

This paper presents Lethe, a database-native deletion layer for RAG knowledge bases built on PostgreSQL with the pgvector extension [4]. Lethe treats deletion as a first-class, provable operation. Our contributions are:

1) **Partition-scoped hard deletion across every index.** The vector table is partitioned by tenant. A deletion removes the row in the same transaction as its audit entry and then, after the commit, runs `VACUUM FULL` on the affected partition, which rewrites it into a new file and rebuilds both its HNSW and its IVFFlat index, so the work is bounded by one tenant's data.
2) **A signed, hash-chained audit log with tested gap detection.** Every deletion is recorded with an ECDSA signature over a message that includes the previous entry's content hash. We confirm experimentally that removing an entry from the middle of a chain segment breaks the link of the entry after it.
3) **Subject-level cascading erasure with a certificate.** An erasure request names a person, not a document. Lethe hard-deletes every document of that subject across all partitions and issues a deletion certificate that a third party can verify entry by entry.
4) **A storage-level erasure test and a measured evaluation.** We search the database's heap, index and write-ahead-log files for a deleted embedding's exact bytes and compare a plain deletion with three versions of

Lethe's deletion path. The test showed that rebuilding the indexes inside the deleting transaction left copies behind, that doing it after the commit cleared the indexes but not the heap, and that the final path, `DELETE` plus `VACUUM FULL`, cleared every data file we searched. We also measure how long each path stalls concurrent queries, latency, rebuild cost against an unpartitioned baseline at 2 to 16 tenants, agreement with exact search, and gap detection in the audit chain.

The rest of the paper is organised as follows. Section II reviews related work, Section III describes the system and methodology, Section IV reports results, Section V discusses limitations, and Section VI concludes.

## II. Related Work

Machine unlearning research has mostly targeted a model's own parameters. Sharded training schemes such as SISA retrain only the shard that held the removed data [5], and algorithms for specific model families remove a point's influence without full retraining [6]. For RAG, forgetting can instead be pursued at the knowledge-base level: Wang et al. modify the external knowledge base so that the system behaves as if data were forgotten, without interacting with the language model directly [7]. These approaches address what the model or the retrieval pipeline reveals, but they do not state whether the stored embeddings are physically gone.

That storage question is what the more recent vector-database analysis raises [3]. It showed that soft-deleted embeddings remain reconstructible from the raw index files of several HNSW implementations, and it proposed a cryptographically signed proof of deletion. It stops at demonstrating the weakness and a basic proof. It does not give an efficient way to remove data for real, and it does not measure what that costs.

The cost is rooted in how the indexes work. HNSW stores vectors in a layered proximity graph [8], while IVF groups vectors into clusters and can be combined with product quantization [9]. Deleting a vector from either structure while keeping search quality generally requires repairing or rebuilding the structure, which is why many systems defer to a flag. Lethe limits that rebuild to one partition rather than avoiding it. A second source of residue is the storage engine itself: PostgreSQL's `DELETE` only marks a heap tuple dead, and the space is reclaimed when the table is vacuumed [10].

On the compliance side, benchmarking work on database systems has shown that GDPR features such as timely deletion and audit logging carry a measurable performance cost [11], which motivates measuring our deletion path instead of assuming it is cheap. For the audit record itself, tamper-evident logging is well established: forward-secure hash chains protect log entries against later alteration [12], and history-tree designs allow efficient verification of a log's integrity [13]. Lethe applies the simpler hash-chain idea to deletion records, since the goal is to show that a record was neither altered nor removed.

Table I summarises how this work differs from common practice in the space it sits in.

TABLE I
How Lethe Differs from Common Practice

| Aspect | Common practice | This work |
|---|---|---|
| Delete semantics | Soft delete, embedding stays on disk | Row deleted, partition rewritten with `VACUUM FULL`, residue measured |
| Index rebuild | Whole index, or none | Affected partition only, cost tied to the tenant's own size (index rebuild measured to 16 tenants) |
| Proof of deletion | None, or one signed record | ECDSA-signed, hash-chained log with chain verification |
| Unit of erasure | A single record | A data subject, cascaded across partitions |
| Evidence for regulators | Log lines | Verifiable deletion certificate |
| Evaluation | Functional demonstration | Latency, rebuild cost vs. tenants, byte-level file search, exact-search agreement, gap test |

## III. Proposed System and Methodology

### A. Architecture

Lethe has three layers on top of one PostgreSQL database with the pgvector extension [4]. A foundation layer holds the schema, the tenant partitions and the vector indexes. A logic layer implements hard deletion, signing, verification and cascading erasure. An evaluation layer generates the corpus and measures the system. A small FastAPI service exposes the logic layer to a web dashboard that demonstrates the full flow of search, erasure, audit and verification, including a demo action that corrupts one stored hash so tamper detection can be shown live. Fig. 1 shows the deletion path.

### B. Schema and Partitioning

Four metadata tables sit beside the document store. `data_subjects` records each person or organisation and the tenant that owns their data. `consent_status` records whether consent is held. `deletion_requests` records each erasure request with its status and a due date reflecting the one-month response window of Article 12(3) [2]. `deletion_audit_log` is the proof-of-deletion record described in Section III-D.

The `documents` table stores the text and a 384-dimension embedding per record and is list-partitioned by `tenant_id`, so each tenant's data lives in its own partition. A secondary index on the subject identifier lets the system find all of one subject's documents across partitions. Each partition carries an HNSW index with cosine distance and pgvector's default build parameters, and an IVFFlat index with cosine distance and 10 lists. IVF with product quantization is not supported natively by pgvector and is out of scope. A boolean `deleted` flag exists in the schema but is not used by the hard-delete path.

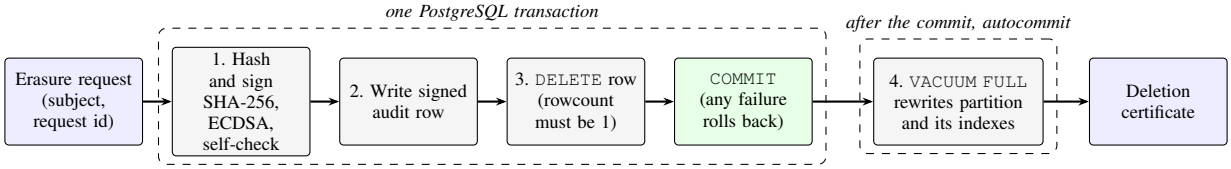



Fig. 1. Deletion path for one document, repeated per document. The signed audit entry and the row deletion commit together or not at all. `VACUUM FULL` then runs as its own statement after the commit, because it cannot run inside a transaction block. It rewrites the partition into a new file and rebuilds both indexes. Earlier versions rebuilt the indexes inside the transaction, which left copies (Section IV-C). A failure after the commit leaves the physical copies in place (Section V).

### C. Hard Deletion

Hard deletion takes a document identifier, its tenant and a deletion request identifier. Inside one database transaction it hashes and signs the document (Section III-D), because the content must still exist to be hashed, writes the audit row, and deletes the document's row from the tenant's partition. The transaction commits only if every step succeeds and the delete affects exactly one row, so an audit entry is never left without its deletion, or the reverse. After the commit, the connection switches to autocommit and runs `VACUUM FULL` on the partition, which cannot run inside a transaction block. `VACUUM FULL` writes the partition's live rows into a new file and rebuilds every index on it, so neither the deleted row's heap bytes nor any index copy of it survives in the old files. Doing this for one partition instead of one table holding every tenant's data is the central cost argument of this work: the work done is bounded by one tenant's data, not by the size of the whole knowledge base. The path went through four versions, and Section IV-C reports the measured effects of the last three. The first rebuilt only the HNSW index and never vacuumed. The second rebuilt both indexes inside the deleting transaction and then vacuumed. The third moved the rebuilds after the commit, and the final version replaced the rebuilds and the plain `VACUUM` with `VACUUM FULL`.

The rewrite is needed for a physical reason. An IVFFlat index stores a copy of each vector inside its own pages, not only a pointer into the heap, so a deleted embedding stays recoverable there until the index is rebuilt, and a plain `DELETE` only marks the heap tuple dead. The first version of our deletion path rebuilt only the HNSW index and never vacuumed, a gap the authors identified in their own audit of the deletion path and closed in later versions.

**What the rewrite is for.** They do not change what an ordinary query returns, because PostgreSQL checks tuple visibility on every index scan, so a stale index never returns an already-deleted row. The exposure at stake is forensic: the embedding's bytes in heap and index pages, recoverable by someone with raw page or disk access. Plain `VACUUM` makes the space of a dead tuple reusable but does not promise to overwrite it, whereas `VACUUM FULL` rewrites the table into a new file and rebuilds its indexes, but holds an exclusive lock on the whole partition while it runs [10] and needs temporary disk space for the second copy. The earlier versions used `REINDEX`, which without `CONCURRENTLY` also locks out readers of the index it rebuilds [14]. Section IV-C measures what each version leaves behind and what it costs, and Section IV-F measures how long each stalls readers.

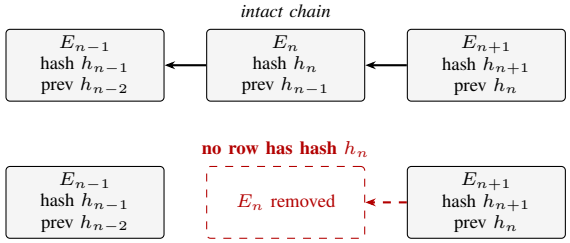



Fig. 2. Each entry's signed message includes the content hash of the entry before it. Altering $E_n$ breaks its signature check. Removing $E_n$ leaves $E_{n+1}$ pointing at a hash that no row holds. Removing the newest entry is the one change this check cannot see.

### D. Signed, Hash-Chained Audit Log

For each deletion, Lethe computes a SHA-256 hash of the document content [15]. It builds a message from the document identifier, the content hash, a UTC timestamp and the content hash of the previous audit entry, and signs that message with ECDSA over the SECP256R1 curve [16]. The first entry uses a fixed all-zero genesis value in place of a previous hash. The content hash, signature, timestamp and previous hash are stored in `deletion_audit_log`. Immediately after signing, the signing function verifies its own signature against the same in-memory message and refuses to write the row if that check fails, a guard added after the anomaly described in Section V.

Because each signed message contains the previous entry's content hash, the log protects against two different attacks (Fig. 2). Altering one entry breaks that entry's signature check. Deleting an entry from the middle breaks the link, because the next entry's stored previous hash no longer matches the entry before it. Verification therefore has two forms: a per-entry signature check, and a walk of the whole table in timestamp order that checks every link and every signature starting from the genesis value. The walk returns a single boolean and stops at the first break it finds.

### E. Cascading Subject Erasure and Certificate

A real erasure request names a person. Given a subject and a request identifier, Lethe looks up every document of that subject across all partitions, hard-deletes them one after

another, each with its own signed audit entry and its own `VACUUM FULL`, and marks the request completed only after all of them have succeeded. A deletion certificate can then be assembled. It contains the subject identifier, the request time, the completion time (the time of the last audit entry), and the list of audit entries with document identifier, content hash, previous hash, signature and timestamp, so that a third party can verify each entry independently. The timestamp is emitted as the exact string that was signed, because any reformatting makes a valid signature fail verification (Section V).

### F. Dataset and Embeddings

No public corpus mixes operational security records with personal identifiers, so we generate a synthetic security-incident corpus. Four sentence templates modelled on real log lines (failed logins, suspicious outbound traffic, flagged processes, unusual cloud-storage access) are filled with randomly chosen first names, IP addresses, hostnames, counts and timestamps from the previous 90 days. Each record is embedded with the all-MiniLM-L6-v2 sentence-transformer model [17], which outputs 384-dimension vectors. Records are spread round-robin across two tenants. The generator is not seeded, so every run draws a fresh corpus, and the timing measurements are taken on the partition of a single tenant.

### G. Evaluation Design

We run seven measurements. *Deletion latency* is timed for a single hard delete and for a cascading erasure of 50 documents belonging to one subject, and, for both paths, end to end at four partition sizes. *Rebuild cost* is the time to `REINDEX` the partition's HNSW index at 100, 500, 1,000 and 5,000 documents. It is compared with one HNSW index over both tenants at the same per-tenant volume ($2N$ rows when a partition holds $N$), and, to test the effect of tenant count, with one combined index over 2, 4, 8 and 16 tenants of uneven sizes (100 to 1,800 documents each) while the tenant under test stays at 500 documents. These comparisons use standalone tables built for the experiment, not real partitions, and each tenant-count configuration was repeated three times.

*Storage erasure* copies the heap file and both index files of the partition out of the database container after a checkpoint and searches their bytes for the deleted embedding, which pgvector stores as 384 consecutive little-endian 32-bit floats (1,536 bytes), and for a 64-byte fragment of it. We run 20 documents through each of three deletion paths (index rebuilds inside the deleting transaction, rebuilds after the commit with a plain `VACUUM`, and the final `DELETE` plus `VACUUM FULL`, which we ran both through a harness that repeats its statements and through the shipped deletion routine), fifteen through a plain `DELETE` as a soft-delete baseline (three runs of five), and search the write-ahead-log segments for one document. A flat byte search can miss data in a large index file, so a positive is evidence of presence but a negative is informative mainly in comparison with the same search on another version.

*Path comparison* compares the end-to-end time, including signing and the audit write, of the three-step path (post-commit `REINDEX` of both indexes and a plain `VACUUM`) with that of the final `DELETE` plus `VACUUM FULL` path, three repeats at each of four sizes, and times `VACUUM FULL` alone 20 times at each size.

*Concurrent blocking* runs a background thread that loops the same top-10 similarity search the dashboard uses against the 5,000-document tenant partition while a deletion runs on the main thread. For each design we time the window in which locks are held (the two `REINDEX` statements and the `VACUUM` for the three-step path, the `VACUUM FULL` statement for the final path) and record each query that overlaps it, the baseline query time before the deletion, and the longest overlapping query. A trial counts as blocked if that query took more than three times the baseline. Each design ran five trials with one reading client.

*Retrieval quality* is measured as the overlap between top-10 results before and after hard-deleting 20 random documents (20 query documents, each excluded from its own results), as agreement with exact search (a forced sequential scan, plan confirmed with `EXPLAIN`, with the planner's index recorded), and under a stress test on 2,000-document tables that each hold one index type, so the index is pinned, with `ivfflat.probes` set to 1 and `hnsw.ef_search` to 10. The pairwise cosine similarity of 200 corpus documents quantifies how easy the corpus is.

*Integrity* is checked by verifying the most recent entry, by modifying a stored hash, and by a gap test: four real hard deletions build a disposable four-entry segment, the second entry is deleted outright, and the remaining links are re-checked directly rather than through the global boolean.

The latency, rebuild and overlap suite was run three times on 26 and 27 September 2026 against the first implementation (HNSW rebuild only, no vacuum), and three times on 10 October 2026 against the version that rebuilds inside the transaction. The single-delete and cascading-erasure measurements were run once more on 10 October after the rebuilds moved to the commit's far side. They were not rerun after the switch to `VACUUM FULL`, apart from the single-delete timing in the path comparison, and the retrieval measures were likewise taken on the earlier paths. The matched-volume unpartitioned comparison ran once, the exact-search comparison twice, and the stress test once. Every run is appended to a timestamped CSV. All experiments ran against PostgreSQL 16 with pgvector in the `pgvector/pgvector:pg16` Docker image on a single Windows 11 laptop (12th Gen Intel Core i5-1235U, 10 cores and 12 threads, 16 GB of memory) through Docker Desktop on WSL2, whose virtual machine was allotted 12 CPUs and about 7.6 GB. The data directory was on a Docker named volume, and ordinary background activity was present, so absolute times will differ on server hardware, while the comparisons between designs were run on the same machine.

## IV. Experimental Results

Table II maps each project objective to the evidence that addresses it, so the results below can be read against what they were designed to show.

TABLE II
OBJECTIVE-TO-EVIDENCE MAPPING

| # | Objective | Evidence |
|---|---|---|
| 1 | Per-tenant partitions and deletion metadata in a relational schema | Sec. III-B |
| 2 | Hard deletion that removes the row and rewrites its partition, including every index | Sec. III-C, IV-A, IV-B |
| 3 | Physical erasure measured at the storage level | Sec. IV-C |
| 4 | Signed, hash-chained audit log with tested gap detection | Sec. III-D, IV-E |
| 5 | Subject-level cascading erasure with a certificate | Sec. III-E, IV-A |
| 6 | Cost against an unpartitioned baseline across tenant counts | Sec. IV-B |
| 7 | Retrieval quality after deletion | Sec. IV-D |
| 8 | Evaluation on a synthetic security-incident corpus | Sec. III-F |
| 9 | Availability cost of each deletion design | Sec. IV-F |

### A. Deletion Latency

Table III reports latency before and after closing the implementation gap. A single hard delete took 96.6, 146.5 and 141.5 ms across the three runs of the in-transaction version (mean 128.2 ms), against 64.8, 72.1 and 67.9 ms (mean 68.3 ms) for the first implementation. The two sets are not cleanly comparable, because the partition held 6, 54 and 54 documents in the first runs and 18, 18 and 17 in the later ones. The cascading erasure of the same 50 documents is the cleaner comparison: it rose from 4,441 ms to 5,573 ms, about 25 percent, or from 88.8 to 111.5 ms per document, the direction expected since each deletion now does strictly more work. As a control, the HNSW-only rebuild benchmark, which the later changes did not touch, gave similar means before and after at 100 to 1,000 documents. One further run after the rebuilds moved past the commit gave 132.1 ms for a single delete, on a partition of 57 documents, and 6,118 ms for the cascade (122.4 ms per document). Both lie inside the range of the in-transaction runs, as expected since the change reorders work and does not add any, but the partition was larger than in those runs and one run cannot show a difference of a few percent. All of these used the earlier paths. The final path was timed for a single deletion at four sizes (Table VI: 106 ms at 100 documents, 499 ms at 5,000), but we have not re-timed the 50-document cascade, in which each of the 50 deletions now rewrites the partition, so its cost on the final path is unmeasured.

### B. Rebuild Cost, Partitioned Versus Unpartitioned

Fig. 3 compares the cost of rebuilding one tenant's HNSW index with that of rebuilding a single index over both tenants at the same per-tenant volume. The partitioned rebuild was between 1.97 and 2.35 times cheaper at every size (means of the three runs of the in-transaction version against one unpartitioned run), about the volume ratio of two equal tenants.

TABLE III
DELETION LATENCY OF THE EARLIER PATHS (MEANS, WITH RANGES; "IN TXN" AND "AFTER COMMIT" ARE THE TWO POSITIONS OF THE INDEX REBUILD)

| Measure | First impl. (3 runs) | In txn (3 runs) | After commit (1 run) |
|---|---|---|---|
| Single delete, ms | 68.3 (64.8–72.1) | 128.2 (96.6–146.5) | 132.1 |
| Partition, docs | 6, 54, 54 | 18, 18, 17 | 57 |
| Cascade of 50, ms | 4,441 (4,156–4,766) | 5,573 (5,012–6,290) | 6,118 |
| Cascade per doc, ms | 88.8 | 111.5 | 122.4 |

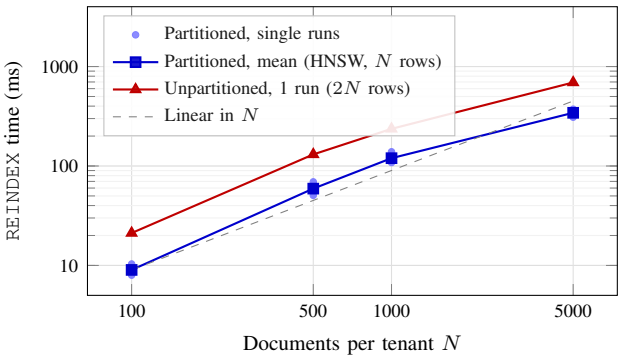



Fig. 3. HNSW index rebuild time by per-tenant volume, on log-log axes. Dots are the three in-transaction-version runs and the square line is their mean. The triangle line is a single combined index over both tenants. The dashed line shows cost proportional to $N$, anchored at the smallest mean.

TABLE IV
REBUILDING ONE 500-DOCUMENT TENANT VERSUS ONE COMBINED INDEX (MEAN OF 3 REPEATS, MS)

| Tenants | Rows | Part. | Unpart. | Ratio | Row ratio |
|---|---|---|---|---|---|
| 2 | 2,000 | 52.1 | 241.1 | 4.6 | 4.0 |
| 4 | 3,500 | 57.0 | 454.0 | 8.0 | 7.0 |
| 8 | 6,100 | 61.8 | 473.6 | 7.7 | 12.2 |
| 16 | 10,000 | 70.9 | 824.9 | 11.6 | 20.0 |

Table IV removes that limitation by varying the number and size of the other tenants while the tenant under test stays at 500 documents.

The partitioned side is the same 500-document table in every configuration, so its flatness (52 to 71 ms) holds by construction. What the experiment adds is the unpartitioned side, whose mean grew from 241 ms at 2 tenants to 825 ms at 16. The ratio grew from 4.6 to 11.6, but at 8 and 16 tenants it stayed below the raw row ratio (7.7 against 12.2, and 11.6 against 20.0). Unpartitioned cost therefore tracks total rows, roughly proportionally or slightly less, and the benefit of partitioning is that a deletion's cost stays tied to the tenant's own size, not that the ratio grows without bound. The measurements are noisy: over three repeats the 16-tenant unpartitioned time ranged from 645 to 1,147 ms and the partitioned time from 48 to 96 ms.

TABLE V
WHERE A DELETED EMBEDDING'S BYTES WERE STILL FOUND (TRIALS OUT OF 15, 20, 20 AND 20)

| File searched | Plain DELETE | Rebuild in txn | Rebuild after commit | Final path |
|---|---|---|---|---|
| Heap, full pattern | 15/15 | 20/20 | 20/20 | 0/20 |
| Heap, 64-byte part | n/m | 20/20 | 20/20 | 0/20 |
| HNSW index | 11/15 | 13/20 | 0/20 | 0/20 |
| IVFFlat index | 11/15 | 13/20 | 0/20 | 0/20 |

n/m: not measured. Final path: DELETE followed by VACUUM FULL, run through the shipped routine.

TABLE VI
PER-DELETION TIME OF THE TWO PATHS, AND VACUUM FULL ALONE (MS)

| Docs | Three-step path | Final path | VACUUM FULL alone |
|---|---|---|---|
| 100 | 101.3 | 105.7 | 37.2 ± 6.7 |
| 500 | 149.9 | 181.5 | 85.4 ± 10.6 |
| 1,000 | 226.7 | 229.6 | 147.1 ± 12.5 |
| 5,000 | 469.0 | 498.7 | 475.2 ± 88.1 |

Means of 3 repeats for the first two columns; mean ± standard deviation of 20 for the last.

Within the partitioned series, a 50-fold increase in size, from 100 to 5,000 documents, raised the cost in the three later runs by 34 to 47 times, close to proportional. An early reading of our notes suggested that cost grows faster than linearly, about 8 times for a 5-fold increase from 1,000 to 5,000 documents. That figure came from a single run (871 ms at 5,000). The other five runs show 2.6 to 3.1 times for the same step, and we could not reproduce the 871 ms result, so the data support roughly linear growth and we withdraw the stronger claim.

### C. Storage-Level Erasure

Table V reports what remained in the database's files. A plain DELETE, the soft-delete baseline, left the embedding in the heap file in all 15 trials and in each index file in 11 of 15 (5, 4 and 2 of 5 in the three runs), so the search finds a live embedding more reliably in the heap than in an index file.

The second version of the deletion path ran its two index rebuilds inside the deleting transaction. After it, the full 1,536-byte pattern was still found in each index file in 13 of 20 trials. A match cannot occur by chance, so these were copies that survived the rebuild. Running the rebuilds as their own statements after the commit reduced this to 0 of 20 for both indexes, confirmed on the production path itself. The difference is significant (hypergeometric $p < 10^{-5}$), and because the same search is the same instrument in both conditions, the contrast is the evidence, not the absolute zero. We have not verified the mechanism, but a rebuild inside the deleting transaction appears to have indexed the just-deleted tuple.

The heap was not helped by that change. After the three-step path the pattern was still in the heap file in all 20 trials, and so was a 64-byte fragment of it: the plain VACUUM that ends that path marks the dead tuple's space reusable but does not overwrite it, so the bytes persist until a later write happens to reuse that page, which is not bounded in time. The final path, DELETE followed by VACUUM FULL, left neither the full pattern nor the 64-byte fragment in the heap, and left the full pattern in neither index, in any of 20 trials, because it writes the table into a new file and rebuilds its indexes. An earlier run of 20 through a harness that repeats the path's statements gave the same result for the full pattern. The write-ahead log also held the embedding (one document, checked before its deletion), and nothing in the deletion path alters the log. The index zeros need a caution: in the latest baseline run the search found a live embedding in an index file in only 2 of 5 trials, so a zero there is weaker evidence than the heap's zero, where the baseline found the pattern every time. The index result rests on the contrast with the in-transaction version and on the mechanism, not on the search alone.

Table VI compares the three-step path with the final path, end to end including signing and the audit write. The final path cost 4, 21, 1 and 6 percent more at 100, 500, 1,000 and 5,000 documents, differences that three repeats cannot separate from run-to-run noise. VACUUM FULL alone took 37 to 475 ms with a spread that grows with size (380 to 667 ms at 5,000 documents), about 1.1 to 1.4 times the mean HNSW-only rebuild at the same sizes from 500 documents upward (the 5,000-document rebuild mean includes one 871 ms outlier). Both paths block some concurrent access while they run, VACUUM FULL on the whole partition and REINDEX on the index it rebuilds [14], and Section IV-F measures it. VACUUM FULL also writes a second copy of the table before dropping the old one, so it needs temporary disk space, which we did not measure.

### D. Retrieval Quality After Deletion

Table VII gives the retrieval measures. Removal alone predicts that, with 20 of 5,000 documents deleted, about 0.8 of the 200 result slots would have held a deleted document and been refilled, an overlap near 0.996. The first implementation matched this in all three runs (0.995). The later version, which rebuilds both indexes inside the transaction, gave 0.91, 0.99 and 0.98. With only 20 query vectors one run is noisy, but the lower and more variable values deserve a second explanation that we have not tested: each of the 20 deletions now rebuilds the IVFFlat index, which re-runs clustering, and a search through a single probed list can then return different neighbours even for unrelated documents. The index the planner used was not recorded in this measure.

Against exact search at default settings, the approximate top-10 matched the true top-10 for all 20 queries in both runs, and the planner chose the IVFFlat index. Two checks show that this reflects an easy corpus. Across 200 corpus documents, the mean pairwise cosine similarity was 0.43, but 4.5 percent of pairs exceeded 0.9 and the maximum was 0.999, because

TABLE VII
RETRIEVAL QUALITY AFTER DELETION AND UNDER STRESS

| Measure (top-10, 20 queries) | First implementation | Later implementation |
|---|---|---|
| Overlap before vs. after deleting 20 docs, per run | 0.995 in all three | 0.91, 0.99, 0.98 |
| Recall against exact search, default settings, per run | not measured | 1.0, 1.0 |
| Index chosen by planner | not recorded | IVFFlat |
| Pinned index, starved settings, 2,000 docs (1 run) | not measured | IVFFlat 1.00, HNSW 0.90 |

documents built from the same template differ only in a name, address or hostname. When each index type was pinned on its own table and starved of search effort, IVFFlat with one probe still reached 1.00 on 2,000 documents, while HNSW with `ef_search` of 10 fell to 0.90. Both are single runs at one size, so which index degrades first should not be read as a general property. The result shows that deletion did not corrupt retrieval here, not that either index is exact.

### E. Integrity Verification

The most recent audit entry passes per-entry signature verification, and modifying a stored hash makes verification of that entry fail. The whole-table check returned *false* on the real audit history before the gap test began, because a demo corruption from 6 October remains in it. Since the walk stops at the first break, a single boolean cannot show that a *new* break has occurred, nor where. For this reason the gap test checks its own disposable segment directly. Before the deletion, all four entries' links and signatures verified. After the second entry was deleted outright, the check on the remaining three failed, because the third entry's previous hash matched no row. The segment was then removed and the real history was left unchanged. This confirms that removal of a middle entry is detected, and it is the one tamper case beyond field mutation that we have tested. We did not measure chain verification time.

### F. Concurrent Blocking

We had assumed that the three-step path, unlike `VACUUM FULL`, never blocks readers, and the first test of that assumption showed it was wrong, which removed the main argument for keeping it. Table VIII reports the five-trial comparison on a 5,000-document partition. Every trial of both designs showed blocking: a query that took about 22 ms with no deletion running took 327 to 376 ms during the three-step path's lock window and 386 to 433 ms during the final path's `VACUUM FULL`, that is roughly 15 and 19 times the baseline. The queries did not fail, they waited and then completed as the lock was released.

TABLE VIII
READER STALL DURING THE LOCKING STATEMENTS, 5,000 DOCUMENTS (MEAN OF 5 TRIALS)

| Measure | Three-step path (`REINDEX` ×2 + `VACUUM`) | Final path (`VACUUM FULL`) |
|---|---|---|
| Trials with a blocked query | 5/5 | 5/5 |
| Lock window, ms | 390.3 | 402.0 |
| Baseline query, ms | 22.5 | 21.6 |
| Longest overlapping query, ms (range) | 347.8 (327–376) | 412.9 (386–433) |
| Queries overlapping the window | 3 | 1 |

The two lock windows are about the same length, 390 and 402 ms. The longest stall differs more: it was about 19 percent longer under the final path, with the ranges not overlapping, which is the price of its complete erasure. The likely reason is that the three-step path takes its locks in three separate statements, so a reader waits for at most part of the window, and plain `VACUUM` does not block reads at all, whereas `VACUUM FULL` holds one lock across the whole rewrite. We did not test that explanation. The numbers are for one reading client on one machine, five trials per design, reads only, and the deletion code in this test repeats the statements of the two paths rather than calling the shipped routine.

## V. DISCUSSION AND LIMITATIONS

**Physical erasure is achieved for the heap and both indexes, not for the write-ahead log.** The final deletion path commits the signed deletion and then rewrites the partition with `VACUUM FULL`. In 20 of 20 trials the embedding and a 64-byte fragment of it were absent from the heap file and the full pattern was absent from both index files, whereas a plain `DELETE` left it in the heap every time. Earlier versions fell short in two different ways: rebuilding the indexes inside the deleting transaction left the embedding in the index files in 13 of 20 trials, and a plain `VACUUM` left it in the heap in 20 of 20. The write-ahead log holds the embedding regardless and nothing in the deletion path alters it, so backups, replicas and log archives need a separate answer. We chose `VACUUM FULL` over the three-step path because in our tests it cost 1 to 21 percent more time and stalled readers about 19 percent longer at worst (Section IV-F), while giving a complete file-level guarantee on every deletion. The tests used one reading client and no write traffic, so a busier system could separate the two paths further.

**The conclusions moved during testing.** The weakness class is the one [3] reports for soft deletion, only narrower: a plain `DELETE` left everything in place. The conclusions themselves moved three times. A single-document check on a small partition suggested clean index erasure, and repeating it over 20 documents on a larger partition reversed that. Moving the rebuild after the commit reversed it again for the indexes but left the heap, and a concurrency test then removed the argument for keeping the three-step path. We report the

sequence because the flat byte search is itself imperfect: it found a live embedding in an index file in 11 of 15 baseline trials, and in only 2 of 5 in the latest run, which we suspect reflects a larger partition but did not test. Our initial suspicion that the stale index could return deleted documents in search results was also wrong, for the visibility reason given in Section III-C, so the residue is forensic, not functional.

**Cleanup is outside the transaction.** Because `VACUUM FULL` runs after the commit, a crash or error between the commit and the rewrite leaves a row that is deleted and logged but still physically present, with no record that cleanup is pending. A certificate for a completed request therefore attests to the signed deletion, not to completed cleanup. A pending-cleanup marker with a retry would close this window, and we have not implemented one.

**Costs we did not measure.** `VACUUM FULL` writes a second copy of the table before dropping the old one, so it needs temporary disk space about the size of the partition, and we did not test that at scale. We also did not time the 50-document cascade on the final path, where each deletion rewrites the partition, and we did not measure `VACUUM FULL` on an unpartitioned table, so the partitioning comparison covers only the `REINDEX` of one HNSW index.

**The cost evidence is bounded.** The tenant-count comparison uses standalone tables, so the flat partitioned cost holds by construction, and it covers HNSW only. The latency comparison before and after the earlier fixes is confounded by partition size, and the cascade timing predates the final path. The comparison of the two paths has three repeats per size, and the blocking test has five trials with one reading client. The corpus is synthetic and templated, limited to a few thousand documents on a single node, with three runs per version on one laptop.

**Retrieval results are narrow.** Perfect agreement with exact search holds for an easy corpus, two runs and whichever index the planner chose. The stress results are single runs, the lower overlap of the later implementation is unexplained, and none of these measures was repeated on the final path. The IVF-PQ variant is not available in pgvector.

**The audit log has its own limits.** The chain detects an altered entry and an entry removed from the middle, but not removal of the newest entries, because nothing outside the database records how long the log should be. The signing key is a local file on the same system as the database, so anyone holding it could rewrite the whole chain. The chain is global and assumes that deletions are signed one at a time, since the previous hash is read from the table at signing time. The verification walk returns only a boolean, which cannot localise a break.

**Two findings from building the system.** First, a value embedded in a signed message must be treated as an exact byte string wherever it is later shown or transmitted. The signing code embeds a timestamp using Python's default string form, while a JSON response uses ISO-8601 with a `T` separator. That one-character difference made a valid signature fail re-verification until the certificate emitted the exact signed string. Second, during a run that wrote 71 signatures, one signature failed verification against its own stored fields while its chain link was intact. We ruled out stale keys, duplicate timestamps and duplicate document identifiers but did not find the cause, and it did not recur across 142 further signatures. Signing now verifies its own signature before the row is written and refuses to write it if the check fails, so a recurrence would stop at the moment of signing.

## VI. Conclusion and Future Work

We presented Lethe, a deletion layer for RAG knowledge bases on PostgreSQL and pgvector that removes a document together with a signed audit entry, rewrites its tenant partition with `VACUUM FULL` so that no old copy of the embedding remains in the heap or either index, records each deletion in a signed, chained audit log, and erases a data subject across all partitions with a verifiable certificate. On a synthetic corpus a single deletion took 106 ms at 100 documents and 499 ms at 5,000, and rebuilding one tenant's HNSW index cost 52 to 71 ms regardless of how many tenants shared the system, against 241 to 825 ms for a combined index. Searching the database's files showed that earlier versions fell short in ways a functional test would not reveal: rebuilding the indexes inside the deleting transaction left copies in the index files in 13 of 20 trials, and a plain `VACUUM` left the heap copy every time. The final path left the embedding in no data file in 20 of 20 trials at a cost similar to the three-step path, and both paths stalled concurrent readers for about 0.4 s at 5,000 documents.

Future work, in order of priority: time the 50-document cascade on the final path and rewrite each partition once rather than once per document; add a pending-cleanup marker and retry for the window after the commit; repeat the blocking test with several clients and with writes, including non-blocking options such as `REINDEX CONCURRENTLY` and `pg_repack`, which we did not test; measure the temporary disk use of `VACUUM FULL` and its cost on an unpartitioned table; improve the index-file search, for example by parsing pgvector's pages; address write-ahead logs and backups, for example with per-tenant encryption keys that can be destroyed; repeat the tenant-count comparison on real partitions; compare each index type against exact search on a larger, non-templated corpus; make chain verification report the first bad entry; and anchor the head of the audit log outside the database, with the signing key held in a key-management service, so that truncation and key compromise become detectable.

## References


[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation)," Official Journal of the European Union, L 119, 2016.

[3] C. Chakraborttii, J. G. Alvarado, S. Abdulofizova, and S. Dwivedi, "Ghost vectors: Soft-deleted embeddings remain reconstructible in HNSW vector databases," arXiv preprint arXiv:2606.18497, 2026.

[4] pgvector contributors, "pgvector: Open-source vector similarity search for Postgres," Software repository, https://github.com/pgvector/pgvector.

[5] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 141–159.

[6] A. Ginart, M. Guan, G. Valiant, and J. Zou, "Making AI forget you: Data deletion in machine learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[7] S. Wang, T. Zhu, D. Ye, and W. Zhou, "When machine unlearning meets retrieval-augmented generation (RAG): Keep secret or forget knowledge?" arXiv preprint arXiv:2410.15267, 2024.

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[9] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.

[10] PostgreSQL Global Development Group, "Routine vacuuming," PostgreSQL 16 Documentation, https://www.postgresql.org/docs/16/routine-vacuuming.html, 2023.

[11] S. Shastri, V. Banakar, M. Wasserman, A. Kumar, and V. Chidambaram, "Understanding and benchmarking the impact of GDPR on database systems," *Proc. VLDB Endowment*, vol. 13, no. 7, pp. 1064–1077, 2020.

[12] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, 1999.

[13] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. 18th USENIX Security Symposium*, 2009, pp. 317–334.

[14] PostgreSQL Global Development Group, "REINDEX," PostgreSQL 16 Documentation, https://www.postgresql.org/docs/16/sql-reindex.html, 2023.

[15] National Institute of Standards and Technology, "Secure hash standard (SHS)," NIST, FIPS PUB 180-4, 2015.

[16] D. Johnson, A. Menezes, and S. Vanstone, "The elliptic curve digital signature algorithm (ECDSA)," *Int. J. Inf. Secur.*, vol. 1, no. 1, pp. 36–63, 2001.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing and Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.